CS 4391 Introduction to Computer Vision
Homework 4

Instructor: Yu Xiang

April 1, 2026

Download the homework4_programming.zip file from eLearning, Assignments, Homework 4. Finish the following programming problems and submit your scripts to eLearning. You can zip all your edited files for submission. TA will run your scripts to verify them.

It is recommended to use Anaconda `https://www.anaconda.com/` to build a python environment. For example, to create an environment named "cs4391":

- conda create -n cs4391 python=3.10
- conda activate cs4391

Install the Python packages needed by

- `pip install -r requirement.txt`

Install PyTorch (CUDA 13.0 example):

- `pip install torch torchvision --index-url https://download.pytorch.org/whl/cu130`

Make sure the CUDA version matches your system. Check available versions at: `https://pytorch.org/get-started/locally/`

If you don't have a NVIDIA GPU with CUDA, you can install a cpu version.

Here are some useful resources:

- Python basics `https://pythonbasics.org/`
- Numpy `https://numpy.org/doc/stable/user/basics.html`
- OpenCV `https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html`
- PyTorch `https://docs.pytorch.org/tutorials/beginner/basics/intro.html`

Problem 1

(4 points) Understanding of the ViT model.

Run the following script:

```
python hw4_vit_understanding.py
```

The script prints the shapes of tensors at different stages of the ViT pipeline.

1.1 Questions

Answer the following questions. You must refer to the printed outputs from your script.

1. What layer is used for patch embedding in the ViT model?
2. For an input image of size 224×224 and patch size 16×16, how many patches are generated? Show your calculation.
3. The tensor shape changes from $(B, 196, D)$ to $(B, 197, D)$ after adding the CLS token. Explain why the number of tokens increases from 196 to 197.
4. What is the shape of the CLS token after the transformer encoder? Explain how this shape is obtained from the tensor before extraction.
5. What does the final linear layer (classification head) do? Explain its input and output dimensions.
6. Why do we use only the CLS token (index 0) for classification instead of all tokens?

1.2 Submission

Upload the following:

- A screenshot or copied terminal output from running the script
- A PDF or text file containing your answers to the questions

Problem 2

(6 points) Image classification with ViT

2.1 Download the dataset

This is the same dataset used in Homework 2. If you already have it, no need to download again.

We will use the objects in the FewSOL dataset `https://irvlutd.github.io/FewSOL/` for classification. Check the following link:

- `https://utdallas.app.box.com/folder/166466844982?v=FewSOL-Dataset`

Download the real_objects.zip file to the folder "homework4_spring26". Unzip the file.

2.2 Generate a split of the dataset

First, we will use the following script to generate a split of the dataset. Run the python script:

- `python make_split.py --root real_objects --shots 5 --val 2 --test 2`

  `--seed 0 --out split.json`

The dataset will be split into a training set, a validation set, and a test set. The split information will be save into a "split.json" file.

2.3 ViT training for image classification

In this part, you will complete a training script for image classification using a pretrained Vision Transformer (ViT).

The starter code hw4_train_vit.py is provided. You need to complete part of the model construction.

Model construction In the function `build_vit_model`, implement the following:

- Load the ViT-B/16 model using `torchvision.models.vit_b_16`
- Use ImageNet pretrained weights when `-pretrained` is specified
- Replace the final classification head so that the output dimension matches the number of classes in the dataset

The `freeze_backbone` functionality is already implemented for you.

Training After completing the TODOs, run the following required experiment:

```
python hw4_train_vit.py --split split.json --pretrained --freeze_backbone \
    --epochs 5 --save_best --tag vit_freeze
```

This command trains only the classification head while keeping the ViT backbone fixed.

If you have access to a GPU, you may also run the following optional experiment:

```
python hw4_train_vit.py --split split.json --pretrained \
    --epochs 5 --save_best --tag vit_ft
```

This command fine-tunes the entire ViT model.

Questions Answer the following questions based on your implementation and results:

1. Why do we replace the final classification head when training on a new dataset?
2. Which parameters are updated when `-freeze_backbone` is used?
3. What is the best validation accuracy of your pretrained ViT with frozen backbone?
4. What is the test accuracy of your best checkpoint?

Submission Upload the following:

- `hw4_train_vit.py`
- the training curve image (`training_curves.png`)
- a PDF or text file containing your answers

Do not submit the dataset to eLearning. That is too large.